

CSE 4540 Web Programming Lab 03 - Backend Basics

Farzana Tabassum

December 26, 2025

You are required to work with the code provided in the previous theory class and make the necessary modifications. For each schema, insert at least five sample input records. After completing the implementation, export all Postman API collections as a JSON file.

Organize all required project files into a single folder, excluding the *node_modules* directory. Create a GitHub account (if you do not already have one), push the project files to a GitHub repository, and finally share the repository link in the classroom.

1 Introduction

This document outlines the extension of the current Workout CRUD application into a comprehensive **Gym Management System**. The system introduces three interconnected models: *Workout*, *Trainer*, and *User*.

2 Model Specifications & Basic CRUD

2.1 User Model (*userModel.js*)

Represents a gym member/customer.

Fields:

- *name* (String, required)
- *email* (String, required, unique)
- *age* (Number, min: 16, max: 100)
- *membershipType* (String, enum: ['basic', 'premium', 'elite'], default: 'basic')
- *joinDate* (Date, default: Date.now)
- *active* (Boolean, default: true)

CRUD Operations:

- Create new user
- Get all users (with optional filters: active status, membership type)
- Get single user by ID
- Update user (change membership, deactivate account)
- Delete user

2.2 Trainer Model (`trainerModel.js`)

Represents a fitness instructor. **Fields:**

- *name* (String, required)
- *email* (String, required, unique)
- *specialization* (Array of Strings, enum: ['yoga', 'cardio', 'strength', 'pilates', 'cross-fit'])
- *experienceYears* (Number, min: 1)
- *hourlyRate* (Number, required, min: 10)
- *available* (Boolean, default: true)
- *certifications* (Array of Strings)

CRUD Operations:

- Create new trainer
- Get all trainers (with optional filters: specialization, availability)
- Get single trainer by ID
- Update trainer (change hourly rate, availability, add certifications)
- Delete trainer

3 Complex Functionalities

You can add/modify the schema if needed.

3.1 Workout Scheduling System

A user with premium/elite membership can schedule a workout session with a trainer.

Process Flow

- User selects a workout type (from existing workouts or custom)
- User selects an available trainer (based on specialization and availability)
- System checks:
 - If trainer is available at requested time. (bonus)
 - If user has valid membership (premium/elite).
 - If user has no conflicting scheduled workouts. (bonus)
- Creates a scheduled workout with:
 - User ID
 - Trainer ID
 - Scheduled date/time
 - Workout Types
 - Status: ‘scheduled’, ‘completed’, or ‘cancelled’ [initially it’ll be ‘scheduled’]
 - Notes/requirements

3.1.1 Workout Scheduling Flow

1. User submits scheduling request
2. System validates user membership
3. Checks trainer availability
4. Verifies no scheduling conflicts
5. Creates workout record
6. Sends confirmation notification
7. Updates trainer’s schedule